

2026 牛客暑期多校训练营 #3 题解

A. 比特掩码

关键观察

把一个数的二进制位从低到高记为 $b_0, b_1, \dots, b_{29}$ ，再额外规定 $b_{30}=0$ 。

一个连续的 1 段一定在某个位置结束。也就是说，如果第 i 位是 1，并且第 $i+1$ 位是 0，那么这里正好贡献一个 1 段的结尾。反过来，每个 1 段也恰好有一个这样的结尾。

因此

$$f(x) = \sum_{i=0}^{29} [\text{bit}_i(x)=1 \text{ and } \text{bit}_{i+1}(x)=0]$$

注意第 30 位是人为补上的 0，它用来统计最高位处结束的 1 段。例如 1011 的第 1 位和第 3 位都是 1 段结尾，所以答案是 2。

题目要求的是所有数的 $f(a_i)$ 之和，所以只要知道每一对相邻位 $(i, i+1)$ 中，有多少个数满足 $(\text{bit}_i, \text{bit}_{i+1})=(1, 0)$ ，就能得到答案。

维护的信息

对每个 $i=0..29$ ，维护四个计数：

```
cnt[i][0][0]
cnt[i][0][1]
cnt[i][1][0]
cnt[i][1][1]
```

其中 $\text{cnt}[i][p][q]$ 表示当前所有 a_j 中，有多少个数满足二进制的第 i 位是 p ，第 $i+1$ 位是 q ：

$$\text{bit}_i(a_j)=p, \text{bit}_{i+1}(a_j)=q$$

于是当前答案就是：

$$\text{ans} = \sum_{i=0}^{29} \text{cnt}[i][1][0]$$

初始化时，枚举每个 a_j 和每个 i ，把对应的二元状态计数加一即可。复杂度是 $O(30n)$ 。

一次操作如何更新

一次操作会同时作用在所有数上，但每一位的变化只和这一位本来的值、操作类型、以及 x 的这位有关。

设原来某一位是 v ， x 的这位是 c ：

```
type=1: v -> v & c
type=2: v -> v | c
type=3: v -> v ^ c
```

考虑相邻位 $(i, i+1)$ 。如果某个数原来的状态是 (p, q) ，那么操作后它会变成：

```
new_p = transform(p, type, bit_i(x))
new_q = transform(q, type, bit_{i+1}(x))
```

所以对固定的 i ，只需要把原来的四种状态重新映射到新的四种状态即可：

```
tmp[new_p][new_q] += cnt[i][p][q]
```

四种状态都处理完后，用 `tmp` 覆盖 `cnt[i]`。

每次操作要处理 30 组相邻位，每组只有 4 种状态，所以单次复杂度是 $O(30 \times 4)$ ，可以通过题目。

B. 再买一瓶

设中奖概率为 $p=a/b$ ，未中奖概率为 $q=1-p=(b-a)/b$ ，以下所有除法都在模 998244353 的意义下进行。

先确定中奖次数

若在恰好喝完 m 瓶时一共中了 w 次奖，则手中剩余的钱为

$$n - m + cw$$

题目要求此时恰好花光，因此必须有

$$m - n = cw$$

所以当 $m < n$ 或 $(m - n)$ 不能被 c 整除时，答案为 0。否则中奖次数已经唯一确定：

$$w = (m - n) / c$$

接下来只需统计：在长度为 m 、恰有 w 个中奖的所有结果序列中，有多少个不会在第 m 瓶之前把钱花光。

用循环移位计数合法序列

把中奖记为 W ，未中奖记为 L 。喝完一瓶后，手中可继续购买的饮料数的变化量为：

$W: c - 1$
 $L: -1$

记原序列的前缀和为 S_t 。喝完前 t 瓶后，剩余饮料数为

$$A_t = n + S_t$$

一个序列合法，当且仅当：

```
A_t > 0 (0 ≤ t < m)
A_m = 0
```

由 $cw = m - n$ 可知总变化量 $s_m = -n$ ，故第二个条件自动满足。

将序列反转并把每一项取相反数。新序列的两种步长为：

```
原 W -> 1 - c
原 L -> 1
```

它们均不超过 1，总和为 n 。新序列的前缀和 R_k 满足

```
R_k = A_{m-k}
```

因此原序列合法，当且仅当新序列的所有非空前缀和均为正。

现在可应用 **Raney 引理**：若整数序列每一项不超过 1，总和为正整数 n ，则它的 m 个循环移位中恰有 n 个移位的所有非空前缀和为正。

长度为 m 、恰有 w 个中奖位置的序列总数为 $C(m, w)$ 。对每个序列考虑全部 m 个切分位置，根据 Raney 引理，其中恰有 n 个切分后得到合法序列；反过来，每个线性序列恰被计入 m 次。故合法序列数为

```
n / m * C(m, w)
```

每个合法序列的概率都为 $p^w q^{(m-w)}$ ，最终答案为

```
n * inv(m) * C(m, w) * p^w * q^(m-w)
```

实现与复杂度

预处理 $0 \dots \max(m)$ 的阶乘和逆阶乘，即可在 $O(1)$ 求组合数。单组数据用快速幂求模逆与两项幂。

```
预处理: O(max m)
单组: O(log MOD)
空间: O(max m)
```

其中 $\max m \leq 2 \cdot 10^6 < 998244353$ ，阶乘均可逆。

C. 蛋糕店

记一个订单为二元组 (t_i, x_i) 。

删除不会影响答案的订单

先按 t 从小到大排序；若 t 相同，按 x 从小到大排序。

如果两个订单 i, j 满足 $t_i \leq t_j$ 且

$$x_j - x_i \geq t_j - t_i,$$

那么任意一次送达订单 j 的行程都可以顺路送达订单 i 。这是因为这次出发时间一定不早于 t_j ，因此订单 i 已经到达；并且若在同一次行程中送二者，有

$$(d + x_j - t_j) - (d + x_i - t_i) = x_j - x_i - (t_j - t_i) \geq 0.$$

也就是说，订单 i 的等待时间不会超过订单 j 。所以订单 i 可以直接删除。

排序后从后往前扫描，维护已经看到的最大 $x - t$ 。若当前订单的 $x - t$ 不大于这个最大值，它就会被后面的某个订单支配，可以删除；否则保留它。删除后，剩余订单满足 t 严格递增，且 $x - t$ 严格递减。

下面只考虑删完后的订单，并重新编号为 $1, 2, \dots, m$ 。

二分答案

二分最大等待时间 W 。

如果订单 i 在某次行程的出发时间为 d ，那么它会在 $d + x_i$ 被送达。要满足等待时间不超过 W ，必须有

$$t_i \leq d \leq t_i + W - x_i.$$

记 $l_i = t_i$ ， $r_i = t_i + W - x_i$ 。若 $W < x_i$ ，显然不可行。

由于删完后 t_i 递增、 $x_i - t_i$ 递减，所以 l_i 与 $r_i = W - (x_i - t_i)$ 都递增。

为什么可以按前缀分批

关键结论：若固定一个可行的 W ，则存在一种可行方案，使得每一趟送完并返回后，已经送达的订单集合总是删后序列的一个前缀。

证明思路如下。考虑任意一个存在“逆序”的可行方案：某一趟已经送了订单 j ，但某个 $i < j$ 还没送。令订单 j 所在行程的出发时间为 d_1 ，订单 i 之后所在行程的出发时间为 d_2 ，则 $d_1 < d_2$ 。

若 $x_i \leq x_j$ ，把订单 i 加入送 j 的那趟行程不会增加那趟的最远距离，且订单 i 会被更早送达，因此方案不会变差。

否则 $x_i > x_j$ 。若送 j 的那趟行程本来就走到至少 x_i ，同样可以把 i 加入那趟。剩下的情况是，送 i 的那趟最远距离至少为 x_i ，而送 j 的那趟最远距离小于 x_i 。此时如果把 j 改到送 i 的那趟，最远距离不会增加；又因为删后序列满足

$$x_i - t_i > x_j - t_j,$$

所以同一出发时间下，订单 j 的等待时间小于订单 i ，不会破坏 W 的限制。

唯一可能阻碍这个交换的是订单 j 在送 i 的那趟出发时尚未到达，但订单 j 原本已经在更早的那趟出发时到达，矛盾。因此总能消去一个逆序。不断交换即可得到按前缀分批的可行方案。

动态规划判定

定义 dp_i 表示已经送完订单 $1 \sim i$ 并回到蛋糕店的最早时间。初始 $dp_0 = 0$ 。

若最后一趟负责订单 $i + 1, \dots, j$ ，它的出发时间必须不早于 dp_i ，也不早于 t_j ，因此最早出发时间是

$$\max(dp_i, t_j).$$

这趟行程要同时满足所有订单的右端点限制。由于 r 递增，最紧的限制是 r_{i+1} 。于是转移合法当且仅当

$$\max(dp_i, t_j) \leq r_{i+1}.$$

合法时有

$$dp_j = \min_i \left(\max(dp_i, t_j) + 2 \max_{k=i+1}^j x_k \right).$$

如果最终 dp_m 有限, 则 W 可行。

优化转移

直接转移是 $O(m^2)$ 的。下面说明如何做到 $O(m \log m)$ 判定。

首先, dp_i 单调不降。直观上, 送完更多订单再回到蛋糕店不会更早; 也可以从前缀分批 DP 的最优方案中删掉最后若干订单得到证明。

对固定的 j , 合法决策点需要满足两件事:

1. $r_{i+1} \geq t_j$ 。
2. 若 dp_i 将作为实际出发时间, 还要保证 $dp_i \leq r_{i+1}$ 。

因为 r 单调, 可以用一个指针维护第一类条件的左边界。

把转移分成两类:

第一类是 $dp_i \leq t_j$ 。此时出发时间就是 t_j , 只需要在所有合法的这类 i 中选最靠右的一个, 因为区间 $i+1, \dots, j$ 越短, 最大 x 不会变大。用 `set` 维护满足 $dp_i \leq r_{i+1}$ 的合法决策点, 再配合指针即可找到这个最靠右的 i 。

随着 j 增加, t_j 单调不降, 合法决策点的左边界也单调右移, 所以第一类选出的最靠右决策点不会向左移动。于是对应区间 $[i+1, j]$ 的左右端点都只会右移, 可以用单调队列维护其中的最大 x 。

第二类是 $dp_i > t_j$ 。此时转移值为

$$dp_i + 2 \max_{k=i+1}^j x_k.$$

随着 j 增加, 每个决策点对应的后缀最大值只会被更大的 x_j 覆盖。用单调栈维护这些后缀最大值相同的连续区间; 当新的 x_j 覆盖若干栈顶区间时, 对对应区间的转移值做区间加法。线段树维护所有当前决策点的最小转移值。由于每个区间进栈、出栈各一次, 总复杂度为 $O(m \log m)$ 。

二分 W 的上下界可取:

$$\max x_i \leq W \leq \max t_i + \max x_i.$$

右端点对应“等所有订单到达后一次性全部送出”的方案。

复杂度

每次判定复杂度为 $O(m \log m)$, 二分 $O(\log 10^9)$ 次。

总复杂度为

$$O(n \log n + n \log n \log 10^9),$$

内存复杂度为 $O(n \log n)$ 。

D. 最长的连续的一

D. 最长的连续的一

题意

给定一个 n 个点 m 条边的 DAG，每个顶点标号为 $a_i \in \{0, 1\}$ 。在所有拓扑序中，求标号序列里极长连续全一段的最大长度 L ，并输出一个达到 L 的拓扑序。

题解

核心观察：拓扑序中一段连续的顶点构成一个“序凸”集合——若 u 和 w 都在该段内，且 u 能到达 w ，则 u 到 w 路径上的所有顶点也必须在该段内（否则拓扑序不合法）。因此，我们要找的便是最大的、仅由标号 1 的顶点组成的、能作为拓扑序中连续段的顶点集合。

三组划分建模：将每个顶点 v 分配到三个组之一：

- 组 0：连续段之前
- 组 1：连续段内部（必须 $a_v = 1$ ）
- 组 2：连续段之后

合法性等价于：对每条边 $u \rightarrow v$ 都有 $\text{grp}(u) \leq \text{grp}(v)$ ，且 $a_v = 0$ 的顶点不能分到组 1。目标最大化组 1 的顶点数。

阈值编码转为最小割：令 $b_v = [\text{grp}(v) \geq 1]$ ， $c_v = [\text{grp}(v) \geq 2]$ ，则 $[\text{grp}(v) = 1] = b_v - c_v$ 。约束转化为蕴含关系：

- $\text{grp}(u) \leq \text{grp}(v)$ (边 $u \rightarrow v$) : $b_u \Rightarrow b_v, c_u \Rightarrow c_v$
- $c_v \Rightarrow b_v$ (组 2 必在组 1 中)
- $a_v = 0$ 时 $b_v \Rightarrow c_v$ (标号 0 的点不能单独停在组 1)

最大化 $\sum(b_v - c_v)$ ，其中 b_v 权为 $+1$ 、 c_v 权为 -1 ，这是一个**最大权闭合子图**问题，用最小割求解：

- $s \rightarrow b_v$ ，容量 1 (正权)
- $c_v \rightarrow t$ ，容量 1 (负权的绝对值)
- $c_v \rightarrow b_v$ ，容量 ∞
- 边 $u \rightarrow v$: $b_u \rightarrow b_v, c_u \rightarrow c_v$ ，容量 ∞
- $a_v = 0$: $b_v \rightarrow c_v$ ，容量 ∞

答案 $L = n - \text{maxflow}$ (最大权闭合子图的权值 = 正权总和 - 最小割)。

构造方案：从 s 出发在残量网络上 BFS 得到源点侧，由此读出每个顶点的 $\text{grp}(v) = [\text{在源点侧}(b_v)] + [\text{在源点侧}(c_v)]$ 。然后以 grp 为首要关键字做拓扑排序 (组 0、组 1、组 2 依次输出)，即可得到一个最优拓扑序。

复杂度：所有源点边容量为 1，故最大流不超过 n ，Dinic 最多增广 n 次，每次 $O(n + m)$ ，总计 $O(n(n + m))$ 。

E. 扫雷

题目回顾

给定一个整数 x ，要求构造一个扫雷局面，使得这个局面恰好有 x 种可能的地雷分布。

局面由数字和 * 组成：

- * 表示未翻开的格子，它可以有雷，也可以没有雷。
- 数字 0 到 8 表示已经翻开的安全格，它本身没有雷。
- 每个数字必须等于它周围八个方向相邻格子中的雷数。
- 只有 * 格子可以放雷。

本题和真实扫雷游戏有一点不同：总雷数不固定。也就是说，两种合法地雷分布可以有不同数量的雷。

输出的局面中，* 的数量不能超过 40。输入范围为：

$$1 \leq T \leq 10, \quad 1 \leq x \leq 300.$$

我们需要对每个 x 构造一个合法局面。

前言

这题预期是一道比较Heuristic的题目，* 的数量不超过40是一个很宽松的限制，有很多种方法都可以给出合法构造。并且由于x只有1~300这些情况，即使是比较慢的构造方法，也可以通过打表通过此题。

下面提供一种可行思路做抛砖引玉。验题人另外给出了几种不同做法，由于篇幅原因不一一介绍。

核心思想：两行条带构造

我们只输出形如下面这样的局面：

```
*****
p
```

其中第一行全是 *，第二行是一个字符串 p 。字符串 p 的每个字符可以是数字，也可以是 *。

例如：

```
*****
*1*2*
```

对于每一列：

- 如果第二行是数字，那么这一列只有上方那个格子是 *。
- 如果第二行也是 *，那么这一列上下两个格子都是 *。

这样做有两个好处：

- 局面非常窄，方便用动态规划计算方案数。
- 我们可以离线搜索出每个 x 对应的短条带，然后在标程中打表直接输出对应条带。

为什么两行条带足够表达很多数

先看几个小例子。

构造 $x = 5$

输出：

```
***
*1*
```

五个*中恰好一个有雷，因此方案数为： $\binom{5}{1} = 5$ 。

构造 $x = 11$

输出：

```
*****
*1*1*
```

有两种情况：最中间的两个*中恰好有一个雷；最中间的两个*中没有雷，两边各有一个雷。方案数为 $\binom{2}{1} + \binom{3}{1}\binom{3}{1} = 11$ 。

可以看到，即使只用两行，也能自然产生一些组合数。把多个数字约束串起来之后，还能得到更多方案数。

如何计算一个条带的方案数

现在我们需要解决一个关键问题：

给定第二行字符串 p ，如何快速计算这个两行条带有多少种合法地雷分布？

因为条带高度只有 2，我们可以从左到右做动态规划。

列状态

每一列最多有两个格子，所以一列中的雷分布可以用一个二进制 mask 表示。

设高度 $H = 2$ ，那么一列状态 s 的范围是：

$$0 \leq s < 2^H = 4.$$

我们约定：

- 第 0 位表示上方格子是否有雷。
- 第 1 位表示下方格子是否有雷。

所以四种状态分别是：

```
0: 00
1: 01
2: 10
3: 11
```


这里的具体上下顺序不重要，只要程序里保持一致即可。

数字列的限制

如果某一列第二行是数字，那么第二行这个格子是已经翻开的安全格，不能有雷。

也就是说，如果当前列字符是数字，则当前列状态中“下方格子”这一位必须为 0。

如果当前列字符是 $*$ ，则这一列上下两个格子都未翻开，当前列状态没有额外限制。

一个数字由三列决定

考虑第二行第 i 列的数字。它周围八个方向相邻的格子，在两行条带中只可能来自下面三列：

- 第 $i - 1$ 列。
- 第 i 列。
- 第 $i + 1$ 列。

因为条带只有两行，数字格自己所在的那个格子不是 $*$ ，也不会放雷。我们在枚举当前列状态时已经保证了这一点。

所以如果第 i 列是数字 d ，那么它是否合法只取决于三列状态：

(第 $i - 1$ 列状态), (第 i 列状态), (第 $i + 1$ 列状态).

只要这三列中有雷的格子总数等于 d ，这个数字就满足。

由于高度是 2，每列最多有两个雷，因此三列最多贡献 6 个雷。不过数字格自己不能有雷，所以实际有用数字只需要考虑 0 到 5。标程的搜索中也是只枚举 0 到 5 和 $*$ 。

条带 DP

我们从左到右扫描条带。

由于判断第 i 列的数字时需要知道第 $i + 1$ 列，所以当我们刚加入新的一列时，能立刻确认的是“上一列”的数字是否满足。

因此 DP 需要记录最后两列的状态。

设：

$$dp[a][b]$$

表示当前已经构造到某一列，倒数第二列状态为 a ，最后一列状态为 b 时的方案数。

转移时，尝试加入新列状态 c 。如果上一列是数字 d ，就检查：

$$\text{popcount}(a) + \text{popcount}(b) + \text{popcount}(c) = d.$$

如果上一列是 $*$ ，则上一列没有数字约束，不需要检查。

然后转移到：

$$newdp[b][c] += dp[a][b].$$

左右边界

左边界可以看作有一列全空状态 0。

初始化第一列时：

$$dp[0][s] = 1$$

其中 s 是第一列允许的状态。

扫描完整个条带后，右边界也看作一列全空状态 0。如果最后一列是数字，就需要用右边界状态 0 再检查一次。

也就是说，最终答案为：

$$\sum dp[a][b].$$

其中，如果最后一列是数字 d ，还要满足：

$$\text{popcount}(a) + \text{popcount}(b) + \text{popcount}(0) = d.$$

如果最后一列是 $*$ ，则直接累加。

搜索所有 $1 \leq x \leq 300$ 的构造

可以把“当前已经构造出的条带”看成一个搜索状态。状态包含：

1. 当前长度。
2. 当前已经使用的 $*$ 数量。
3. 最后一列字符。
4. 当前的 DP 表。
5. 当前条带字符串。

从一个状态出发，尝试在右边添加一个新字符：

- 数字 0 到 5。
- 或 $*$ 。

每添加一个字符，就用 DP 转移更新方案数。

如果当前条带的最终方案数是 v ，并且 $1 \leq v \leq 300$ ，那么我们就记录：

$$ans[v] = \text{当前条带}.$$

只要所有 1 到 300 都被记录下来，就可以停止搜索；并且通过对每一种情况找到解，也完成了正确性的严谨证明。

实际搜索得到的构造中，最多只用了 20 个 $*$ ，最大宽度只有 13。

F. Not Aqre 2

题意

有一个 $n \times m$ 的网格，每个格子填 $0, 1, 2$ 中的一个数，要求任意两个有公共边的格子数字不同。求合法填法数，对 998244353 取模。

限制是 $1 \leq n < 10, 1 \leq m < 998244353$ 。行数很小，列数很大。

矩阵快速幂优化状压dp

如果把一整列的颜色作为状态，那么一列内部相邻格子必须不同，所以合法列数是

$$3 \cdot 2^{n-1}.$$

这个数量最多是 $3 \cdot 2^8 = 768$ 。矩阵快速幂的复杂度为 $O(S^3 \log m)$, S 为矩阵边长。如果直接对这些完整颜色状态做矩阵快速幂， $S = 768$ ，总运算量达到 $1e10$ 以上，无法通过此题。

优化状压

关键观察是：对于下一列来说，当前列的颜色具体名字不重要，真正重要的是当前列内部的“形状”。

考虑一列从上到下的颜色为 $a_1, a_2, \dots, a_n$ 。

由于相邻两个格子不能相同，所以 $a_i \neq a_{i-1}$ 。

当 $i \geq 3$ 时， a_i 有且只有两种可能：

1. $a_i = a_{i-2}$;
2. a_i 是既不同于 a_{i-1} 、也不同于 a_{i-2} 的第三种颜色。

因此，只要知道前两个颜色不同，再用一个二进制位记录每个 $i \geq 3$ 属于哪一种情况，就能唯一确定这一列的相对形状。

定义状态的第 $i - 2$ 位表示：

$$[a_i = a_{i-2}], \quad 3 \leq i \leq n.$$

所以状态数量是

$$2^{n-2}$$

其中 $n = 2$ 时只有一个空状态。

对于一个固定的形状，前两个颜色可以任选两个不同颜色，共 $3 \cdot 2 = 6$ 种。之后每个位置都由形状唯一决定。因此每种形状恰好对应 6 个具体合法列。

特别地， $n = 1$ 需要单独处理。

求转移矩阵

可以分别枚举当前列和下一列的 $3 \cdot 2^{n-1}$ 种情况，并求出当前列的形状 s ，下一列的形状 t 。

如果当前列和下一列可以挨着（每一行的数字都不相同），将转移矩阵 $M_{s,t}$ 加一。

每个 $M_{s,t}$ 的值就是从形状 s 到形状 t 的具体方案数。

矩阵快速幂

第一列可以选择任意形状。每种形状有 6 个具体列，所以初始向量中每个状态的值都是 6。

设状态数为 $S = 2^{n-2}$ 。如果 $m = 1$ ，答案显然是

$$6S = 3 \cdot 2^{n-1}.$$

如果 $m > 1$ ，需要做 $m - 1$ 次列转移。因此答案为

$$\sum_{t=0}^{S-1} \sum_{s=0}^{S-1} 6 \cdot (M^{m-1})_{s,t}.$$

也就是求出 M^{m-1} 后，把矩阵所有元素求和，再乘以 6。

如此优化后，最大矩阵边长是 $2^7 = 128$ ，可以在时间复杂度 $O(S^3 \log m)$ 下通过。

另一种可行思路

本质上方案数也能用一个项数为 $S = 2^{n-2}$ 的线性递推式来表示。

你不需要去手工求解这个线性递推式。如果你意识到了此题是一个项数不多的线性递推式，由于模数是固定的质数，可以先暴力dp出前 $2S$ 项（或更多项），然后利用 Berlekamp-Massey 算法进行求解。

G. 矩阵标记

按数值分别考虑

固定一个数值 x 。若两个值为 x 的格子分别在 $(r1, c1)$ 与 $(r2, c2)$ ，且

$$r1 < r2, \quad c1 < c2$$

它们会标记矩形 $[r1, r2] \times [c1, c2]$ 。不同数值产生的矩形只需取并集，因此可以独立处理每一种数值，最后用二维差分叠加所有矩形。

将 x 出现过的行按升序记为

$$r1 < r2 < \dots < rk$$

在同一行中，只保留 x 出现位置的最小列 $mn[i]$ 与最大列 $mx[i]$ 即可。

相邻出现行之间的分界线

考虑第 rt 行和第 $r(t+1)$ 之间的任意一条横向分界线。一个跨过该分界线的合法矩形，其左上角只能来自分界线上方、右下角只能来自分界线下方。

令

$$L[t]=\min(mn[1],mn[2],\dots,mn[t])$$
$$R[t]=\max(mx[t+1],mx[t+2],\dots,mx[k])$$

若 $L[t]<R[t]$ ，上下两侧确实能各选一个值为 x 的格子作为合法矩形的两个角。并且，所有跨过这条分界线的矩形在相邻两行 $r_t, r(t+1)$ 上的并，恰好可以用一个矩形表示：

行区间 $[r_t, r(t+1)]$ ，列区间 $[L[t], R[t]]$

因此对每个 $t=1..k-1$ ，只要 $L[t]<R[t]$ ，向二维差分加入这个矩形。

正确性说明

对于算法加入的矩形， $L[t]$ 来自某个不晚于 r_t 的出现位置， $R[t]$ 来自某个不早于 $r(t+1)$ 的出现位置。两者列号满足 $L[t]<R[t]$ ，故它确实被一对合法的相等格子所标记，不会多标。

反过来，任取一个由 (ra,ca) 、 (rb,cb) 标记的合法矩形。枚举这两行之间每对相邻的出现行 $r_t, r(t+1)$ ；由于

$$L[t] \leq ca < cb \leq R[t],$$

算法加入的对应小矩形覆盖该高度带内的 $[ca,cb]$ 。这些高度带的并覆盖了 $[ra,rb]$ ，于是原矩形的每个格子都会被算法标记。两边相等，算法得到的正是全部标记格。

时间复杂度： $O(nm)$
空间复杂度： $O(nm)$

H. 季节

序列与多项式的联系

设 S_T 为所有周期属于 $1,2,\dots,T$ 的数组之和所构成的集合。

周期为 p 的序列会被多项式 x^p-1 零化。因此， S_T 中的每个序列都会被下列最小公倍式零化：

$$Q_T(x) = \text{lcm}(x^1 - 1, x^2 - 1, \dots, x^T - 1) = \prod_{d=1}^T \Phi_d(x),$$

其中 Φ_d 为第 d 个分圆多项式。反过来，在该域上，这个条件在有限前缀上刻画了相同的线性空间：当且仅当 Q_T 的系数构成数组 A 的一个合法线性递推时， $A \in S_T$ 。

合法性校验与卷积

设 $D=\deg Q_T$ ，且

$$Q_T(x) = \sum q_j x^j.$$

若 $D<n$ ，需要校验全部递推式：

$$\sum_{j=0}^D q_j A_{k+j} = 0 \quad (0 \leq k < n - D)。$$

这本质上是一次卷积，可用 NTT 高效完成。若 $D \geq n$ ，数组没有额外限制，必然可行。

预处理与二分

由分圆多项式次数的性质，

$$\sum_{d=1}^{573} \varphi(d) \geq 10^5，$$

故只需预处理到约 573。利用

$$x^m - 1 = \prod_{d|m} \Phi_d(x)$$

进行精确除法，可预先求出所需分圆多项式。随着 T 增大，可表示的数组集合单调扩大，因此二分最小可行 T ；每次判定使用一次 NTT 卷积校验。

I. 交换大师

题意

给定数组 $a_1, a_2, \dots, a_n$ ，定义数组的价值为

$$F(a) = \sum_{i=1}^{n-1} |a_i - a_{i+1}|。$$

你最多可以交换一次两个不同位置上的元素，也可以不交换。要求最大化 $F(a)$ 。

约束：

- $1 \leq T \leq 100$
- $2 \leq n \leq 5 \cdot 10^5$
- $0 \leq a_i \leq 10^9$
- 所有测试用例的 n 之和不超过 $5 \cdot 10^5$

答案可能达到 $5 \cdot 10^{14}$ 量级，所以需要使用 `long long`。

基本观察

如果交换两个位置 i, j ，并不是所有相邻差分都会变化。

只有和 i 或 j 相邻的边会变化，也就是这些边：

$$(i-1, i), (i, i+1), (j-1, j), (j, j+1)。$$

不存在的边忽略。

所以对于固定的一对 i, j , 我们可以在 $O(1)$ 时间算出交换前后的变化量。

但是 n 最大是 $5 \cdot 10^5$, 不能枚举所有 $O(n^2)$ 对位置, 因此需要优化。

单独处理相邻交换

如果 i 和 j 不相邻, 即 $i + 1 < j$, 那么 i 附近影响的边和 j 附近影响的边互不重叠, 可以比较干净地拆开分析。

但如果 $j = i + 1$, 边 $(i, i + 1)$ 会同时涉及两个被交换的位置, 公式会重叠, 所以必须单独处理。

因此算法分两部分:

1. 枚举所有相邻交换 $(i, i + 1)$, 直接 $O(1)$ 算变化量。
2. 对所有非相邻交换, 用数学转化在线性时间内求最大值。

最后再和“不交换”的情况取最大值。

局部贡献函数

对于一个位置 p , 定义函数 $g_p(x)$ 表示: 如果把位置 p 的值改成 x , 它和左右邻居产生的贡献是多少。

如果 p 是中间位置:

$$g_p(x) = |x - a_{p-1}| + |x - a_{p+1}|.$$

如果 $p = 1$:

$$g_1(x) = |x - a_2|.$$

如果 $p = n$:

$$g_n(x) = |x - a_{n-1}|.$$

再定义原来的局部贡献为:

$$base_p = g_p(a_p).$$

对于非相邻交换 i, j , 假设 $i + 1 < j$ 。

交换后, 位置 i 放入 a_j , 位置 j 放入 a_i 。

因为两边影响的边不重叠, 所以交换带来的增量为:

$$\Delta = g_i(a_j) - base_i + g_j(a_i) - base_j.$$

目标就是最大化这个式子。

把绝对值拆成直线

先看一个很简单的事实: 如果有一个固定的数 c , 那么 $|x - c|$ 的值只取决于 x 在 c 的左边还是右边。

- 当 $x \geq c$ 时, $|x - c| = x - c$ 。
- 当 $x < c$ 时, $|x - c| = c - x$ 。

也就是说, $|x - c|$ 可以看成两种表达式里较大的那个:

$$|x - c| = \max(x - c, c - x).$$

这两个表达式都是一次函数，也就是直线。

现在回到 $g_p(x)$ 。

$g_p(x)$ 的意思是：如果位置 p 的值变成 x ，它和相邻位置产生多少贡献。

如果 p 是中间位置，它有两个邻居，那么：

$$g_p(x) = |x - a_{p-1}| + |x - a_{p+1}|.$$

这里有两个绝对值。每个绝对值都有两种写法，所以总共最多 $2 \times 2 = 4$ 种组合。

每一种组合展开以后，都会变成一个形如

$$kx + b$$

的表达式。

例如某个邻居是 v ，那么 $|x - v|$ 有两种选择：

- 如果选 $x - v$ ，就相当于给斜率 k 加 1，给常数项 b 加 $-v$ 。
- 如果选 $v - x$ ，就相当于给斜率 k 加 -1 ，给常数项 b 加 v 。

对位置 p 的所有邻居都这样选择一遍，就可以得到若干条候选直线。

因为一个位置最多只有两个邻居，所以候选直线最多只有 4 条。

这些直线的作用是：**以后我们想快速计算 $g_p(x)$ 时，不需要分类讨论如何拆绝对值，只需要在这几条直线里面取最大值即可。**

每个 $g_p(x)$ 是若干个绝对值之和。每个绝对值 $|x - c|$ 都是两条直线的最大值，因此 $g_p(x)$ 也可以表示为有限条直线的最大值。端点最多 2 条，中间点最多 4 条。因此，枚举这些直线不会漏掉 $g_p(x)$ 的真实取值。

直观理解是：

- $g_p(x)$ 是一个由绝对值拼出来的折线函数。
- 折线函数可以看成几条直线拼在一起。
- 我们把这些可能用到的直线都提前列出来，之后做最大值优化。

非相邻交换的优化

对于非相邻交换 i, j ，增量可以拆成两部分：

$$\Delta = g_i(a_j) - base_i + g_j(a_i) - base_j.$$

位置 i 原来放的是 a_i ，贡献是 $base_i$ ；交换后位置 i 放入 a_j ，贡献变成 $g_i(a_j)$ 。所以位置 i 这一侧的变化是 $g_i(a_j) - base_i$ 。位置 j 同理，变化是 $g_j(a_i) - base_j$ 。

非相邻时两边影响的边没有重叠，所以两个变化量直接相加就是总增量。

假设 g_i 选中某条直线：

$$k_i x + b_i.$$

假设 g_j 选中某条直线：

$$k_j x + b_j.$$

那么这一组直线对应的候选增量为：

$$(k_i a_j + b_i - base_i) + (k_j a_i + b_j - base_j).$$

整理一下：

$$\Delta = (b_i - base_i + k_j a_i) + k_i a_j + b_j - base_j.$$

如果直接枚举 i, j ，还是 $O(n^2)$ 。

考虑从左往右枚举右端点 j ，然后枚举 g_j 使用其中的哪一条直线。然后，关键问题在于如何快速找到最好的左端点 i 。

对于左端点 i ，同样可以枚举选择 g_i 中的哪一条直线。选好之后，左端点对答案的影响可以拆成两类信息：

1. 一部分只和旧位置 i 有关，可以在 i 还在左边时提前维护。
2. 一部分还需要乘上当前的 a_j ，所以要按照左端点直线的斜率分类保存。

因此，可以维护 $best$ 数组：

```
best[ki][kj]
```

它表示：

已经在左边、并且可以和当前右端点非相邻交换的位置中，左端点直线斜率为 k_i 时，面对右端点斜率为 k_j ，左端点能贡献的最大历史值。

用公式表示，就是在给定 k_i 和 k_j 的情况下，对于所有可能的左端点 i ，存储 $\max(b_i - base_i + k_j a_i)$ 。

斜率只可能是 $-2, -1, 0, 1, 2$ ，所以 $best$ 只有 5×5 个状态。

为什么要有两个下标？

- k_j 是右端点 j 选择的直线斜率。它会影响历史位置 i 中 a_i 的系数。
- k_i 是左端点 i 选择的直线斜率。它会影响当前值 a_j 的系数。

换句话说：

- 当一个位置 i 被加入历史集合时，我们不知道未来的右端点会选什么斜率，所以把所有可能的 k_i 都预处理一遍。
- 当未来枚举到某个右端点 j 时，可以枚举当前选择哪个 k_i ，然后直接查询对应的历史最优值。

于是从左到右扫描时：

1. 先把已经足够远的位置加入 $best$ 。处理右端点 j 时，只能加入到 $j - 2$ ，这样保证不是相邻交换。
2. 枚举当前 j 的每条候选直线。
3. 根据这条直线的斜率去 $best$ 中查询所有可能的左端点斜率。
4. 用查询到的历史最优值加上当前 j 的贡献，更新答案。

整个过程的本质是：

把“枚举左端点 i ”变成“查询一个 5×5 的最大值表”。

因此每个 j 只需要常数次操作，非相邻交换部分就从 $O(n^2)$ 优化到了 $O(n)$ 。

实现细节

需要实现三个小函数。

第一个是计算当前位置原来的局部贡献：

```
long long base(int p) {
    long long res = 0;
    if (p > 1) res += abs(a[p] - a[p - 1]);
    if (p < n) res += abs(a[p] - a[p + 1]);
    return res;
}
```

第二个是生成 $g_p(x)$ 的所有直线。

做法是收集位置 p 的所有邻居，然后枚举每个邻居选择 $x - v$ 还是 $v - x$ 。

如果一条直线为：

$$kx + b,$$

就把 (k, b) 存下来。

第三个是相邻交换的增量计算。

为了避免边界讨论，可以把所有可能受影响的边编号放进数组，排序去重，然后分别计算交换前和交换后的贡献。

容易写错的点

1. 相邻交换必须单独处理，不能直接套非相邻公式。
2. 端点位置只有一个邻居，生成直线时不要访问越界。
3. 答案和差分和必须用 `long long`。
4. `best` 初始要设成负无穷。
5. 扫描时加入的是 `j-2`，不是 `j-1`，否则会错误包含相邻交换。
6. 最大增量要从 `0` 开始，因为可以不交换。

J. 树.zip

把原树以 `1` 为根。新树中每个点的父亲只能是它在原树中的严格祖先；每条约束 (u, v) 要求 v 仍是 u 在新树中的祖先。目标是最小化新树的深度和。

自底向上传递尚未满足的祖先要求

为每个点 x 维护一个集合 `need[x]`：其中的每个点都必须成为 x 在新树中的祖先。

初始时，每条约束 (u, v) 把 v 放入 $need[u]$ 。随后按原树深度从大到小处理各点。

设当前处理点为 x ：

- 若 $need[x]$ 为空，令 $parent_new[x]=1$ ；
- 否则，取 $need[x]$ 中原树深度最大的点 d ，令 $parent_new[x]=d$ 。删除所有等于 d 的重复要求；把其余要求全部合并到 $need[d]$ 。

为什么可以这样传递？ $need[x]$ 中的点都是 x 的原树祖先，因而在同一条根到 x 的链上。 d 是其中最深的点。若较浅的点 a 也要求成为 x 的新树祖先，而现在把 x 直接挂在 d 下，那么恰好需要 a 成为 d 的新树祖先；这就是把 a 转入 $need[d]$ 的含义。

贪心正确性

当 $need[x]$ 非空、最深要求为 d 时，任意合法方案都必须让 d 成为 x 的新树祖先，所以必有

$$dep_new[x] \geq dep_new[d] + 1$$

直接令 $parent_new[x]=d$ 正好达到这个下界，因此对 x 而言不可能有更浅的选择。

其余要求全部比 d 浅，且都与 d 位于同一条原树祖先链。它们必须在新树中位于 d 之上，转入 $need[d]$ 后恰好保留了这些要求。自底向上处理只会决定当前点指向上方的父边，不会破坏已处理更深点的选择。

故可将任意合法方案逐点替换为上述贪心选择，且深度和不增加；贪心构造最优。

可并堆实现

需要支持：插入一个要求、取出原树深度最大的要求、删除所有相同最大值，以及把一个集合合并到另一个集合。用以原树深度为键的大根左偏堆（或配对堆）即可。

每一条初始约束对应一个堆节点。处理 x 时取堆顶 d ，不断弹出所有值为 d 的节点，再把剩余堆与 $need[d]$ 合并。堆节点总数为 q ，每次合并或弹出为 $O(\log q)$ 。

父亲确定后，再按原树深度从小到大计算

$$dep_new[x] = dep_new[parent_new[x]] + 1,$$

累加即为答案。答案最大为 $O(n^2)$ ，需使用 `long long`。

时间复杂度： $O((n+q) \log q)$

空间复杂度： $O(n+q)$

K. 转向导航

K. 转向导航

题意

给定平面上 n 个整点航点 $P_1, \dots, P_n$ ，汽车依次沿直线段 $P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n$ 行驶。在每个中间航点 P_i ($2 \leq i \leq n-1$)，比较到达方向 $\overrightarrow{P_{i-1}P_i}$ 与离开方向 $\overrightarrow{P_iP_{i+1}}$ ，判断是左转 (LEFT)、右转 (RIGHT) 还是直行 (STRAIGHT)。保证不会掉头。

题解

设到达方向向量 $\vec{a} = P_i - P_{i-1}$ ，离开方向向量 $\vec{b} = P_{i+1} - P_i$ ，则两者之间的转向关系恰好由二维叉积的符号给出：

$$\vec{a} \times \vec{b} = a_x b_y - a_y b_x \begin{cases} > 0 & \Rightarrow \text{LEFT} (\vec{b} \text{ 由 } \vec{a} \text{ 逆时针旋转 } 0^\circ \sim 180^\circ \text{ 得到}) \\ < 0 & \Rightarrow \text{RIGHT (顺时针)} \\ = 0 & \Rightarrow \text{STRAIGHT (同向; 掉头已被输入排除)} \end{cases}$$

直接对每个中间航点计算叉积并输出即可。

精度与溢出注意：坐标绝对值不超过 10^9 ，故方向向量分量不超过 2×10^9 ，叉积不超过约 8×10^{18} ，刚好落在 `long long` (约 9.2×10^{18}) 范围内。因此全程使用**整数运算**，无需浮点数，既无精度误差也无溢出风险。

时间复杂度 $O(\sum n)$ 。

L. 登山对决

L. 登山对决

题意

给定一个 $n \times m$ 的网格，每个格子有一个互不相同的高度。两面玩家交替移动一面旗帜，每次必须把旗帜移到一个四相邻且高度**严格更高**的格子；无路可走的玩家输。给 q 个独立询问，每个询问指定起始格子，判断在最优博弈下先手胜 (First) 还是后手胜 (Second)。

题解

博弈模型：旗帜在网格上移动，只能往更高的格子走，因此所有可能的移动构成一个**有向无环图**（高度严格递增，不可能成环）。这是一个单 token 在 DAG 上的正常博弈 (normal play，无路可走者输)，**不是**多个独立游戏的和，所以不需要 Sprague-Grundy 理论，只需判定每个格子的胜/负状态。

胜负定义 (经典)：

- 一个格子是**必胜** (First)，当且仅当它存在至少一个严格更高的相邻格子，且该相邻格子是**必败**的 (把必败局面扔给对手)；
- 否则它是**必败** (Second)。特别地，局部最大值 (没有更高邻居) 必败。

按高度递减 DP：因为每条边都由低处指向高处，所以按高度**从高到低**处理每个格子时，它的所有严格更高邻居都已被计算完毕。直接迭代：

- 把所有格子按高度降序排序；

2. 依次处理每个格子，检查其四个邻居中是否有“严格更高且必败”的，有则该格必胜，否则必败；
3. 询问 $O(1)$ 查表输出。

复杂度：排序 $O(nm \log(nm))$ ，DP 与询问合计 $O(nm + q)$ 。

M. 漫游者

题意回顾

人在树上从 s 走到 t 。

第一步没有“上一步所在的点”，所以从 s 的所有邻点中等概率选一个。

之后如果当前在 x ，上一点是 y ：

- 如果 x 还有不是 y 的邻点，就在这些点中等概率选一个；
- 否则只能走回 y 。

到达 t 的瞬间停止。要求期望步数，对 998244353 取模。

为什么要用有向边状态

过程需要记住“上一步在哪里”，所以只知道当前点是不够的。

例如当前都在点 x ，但上一步可能来自不同邻点，那么下一步不能走的方向也不同，期望自然可能不同。

因此定义有向边状态：

$$f(u \rightarrow v)$$

表示：当前在 v ，上一点是 u 时，到达 t 的期望剩余步数。

如果 $v = t$ ，已经到终点：

$$f(u \rightarrow v) = 0$$

否则设 v 除了 u 以外还有 k 个邻点。

若 $k > 0$ ，下一步会在这些邻点里等概率选择：

$$f(u \rightarrow v) = 1 + \frac{1}{k} \sum_{w \in \text{adj}(v), w \neq u} f(v \rightarrow w)$$

若 $k = 0$ ，只能走回 u ：

$$f(u \rightarrow v) = 1 + f(v \rightarrow u)$$

直接把所有有向边都列成方程，可以得到正确答案，但高斯消元复杂度为 $O(n^3)$ ，无法通过此题。下面利用树结构在线性时间求解。

以终点为根

把整棵树以 t 为根。

对每个非根点 v ，设它的父亲是 p 。我们关心这条父子边的两个方向：

- $A_v = f(p \rightarrow v)$ ：从父亲走进 v 后的期望；
- $B_v = f(v \rightarrow p)$ ：从 v 走向父亲后的期望。

注意 A_v 只涉及 v 的子树内部，以及一个“向父亲走”的边界值 B_v 。

所以我们希望先求出一个线性关系：

$$A_v = a_v B_v + b_v$$

其中 a_v, b_v 是我们要求的系数。以后只要知道 B_v ，就能立刻算出 A_v 。

后序求系数

按照以 t 为根的树做后序 DP，也就是先处理儿子，再处理父亲。

考虑某个非根点 v ，它有 k 个儿子：

$$c_1, c_2, \dots, c_k$$

叶子情况

如果 $k = 0$ ，从父亲来到 v 之后， v 没有别的邻点，只能走向父亲：

$$A_v = 1 + B_v$$

所以

$$a_v = 1, \quad b_v = 1$$

非叶子情况

下面考虑 $k > 0$ 。

对每个儿子 c_i ，因为儿子已经处理完，我们已经知道：

$$f(v \rightarrow c_i) = a_i f(c_i \rightarrow v) + b_i$$

为了让式子短一点，记：

- $x_i = f(v \rightarrow c_i)$ ：从 v 走进儿子 c_i 后的期望；
- $y_i = f(c_i \rightarrow v)$ ：从儿子 c_i 走向 v 后的期望；
- a_i, b_i ：儿子 c_i 的线性系数。

于是有：

$$x_i = a_i y_i + b_i$$

现在看 y_i 。状态 $y_i = f(c_i \rightarrow v)$ 表示当前在 v ，上一点是 c_i 。

此时不能走向 c_i ，可以走向父亲，或者走向其他 $k - 1$ 个儿子，一共 k 个选择。因此：

$$y_i = 1 + \frac{B_v + \sum_{j \neq i} x_j}{k}$$

这里：

- B_v 是走向父亲后的期望；
- $\sum_{j \neq i} x_j$ 是走向其他儿子后的期望之和；
- 前面的 1 是下一步本身。

接下来把所有 $x_j = a_j y_j + b_j$ 代进去。

令：

$$S = \sum_{j=1}^k a_j y_j, \quad C = \sum_{j=1}^k b_j$$

其中 S 是所有 $a_j y_j$ 的和， C 是所有 b_j 的和。

则

$$\sum_{j \neq i} x_j = S - a_i y_i + C - b_i$$

代回 y_i 的方程，整理得到：

$$y_i = \frac{S + B_v + k + C - b_i}{k + a_i}$$

这个式子很关键：每个 y_i 都只和同一个总量 S 、边界 B_v 有关。

把它再代回 $S = \sum a_i y_i$ ，就能解出 S 。

为了方便理解，记

$$U = \sum_{i=1}^k \frac{a_i}{k + a_i}, \quad V = \sum_{i=1}^k \frac{a_i(k + C - b_i)}{k + a_i}$$

那么

$$S = UB_v + V + US$$

所以

$$S = \frac{UB_v + V}{1 - U}$$

可以看到 S 是关于 B_v 的一次函数。于是每个 y_i 、每个 x_i 也都是关于 B_v 的一次函数。

最后，从父亲走到 v 后，下一步会在 v 的所有儿子中等概率选择：

$$A_v = 1 + \frac{x_1 + x_2 + \cdots + x_k}{k}$$

因此也能得到

$$A_v = a_v B_v + b_v$$

这样就求出了点 v 的系数。

每个儿子只会被统计常数次，所以处理一个点的复杂度是 $O(k)$ 。

自上而下求具体值

后序 DP 结束后，我们知道了每个非根点的 a_v, b_v ，但还不知道每条有向边的具体期望。

接下来从根 t 往下走。

如果 v 是根 t 的儿子，那么

$$B_v = f(v \rightarrow t) = 0$$

因为一旦走到 t ，过程立刻结束。

于是可以算出：

$$A_v = a_v \cdot 0 + b_v = b_v$$

对于其他点 v ，当它的 B_v 已经从父亲那里确定后，仍然使用上一节同样的式子：

$$y_i = \frac{S + B_v + k + C - b_i}{k + a_i}$$

这一次 B_v 是具体值，不是变量，所以可以直接求出每个儿子的

$$B_{c_i} = f(c_i \rightarrow v) = y_i$$

然后继续向下。这样就能得到所有有向边 $f(u \rightarrow v)$ 的值。

计算答案

如果 $s = t$ ，答案是 0。

否则第一步从 s 的所有邻点中等概率选择，并且第一步本身贡献 1。

所以：

$$ans = 1 + \frac{1}{deg(s)} \sum_{v \in adj(s)} f(s \rightarrow v)$$

其中 $deg(s)$ 表示点 s 的度数， $adj(s)$ 表示 s 的所有邻点。

所有除法都在模 998244353 意义下进行，也就是乘以对应数的逆元。

正确性说明

状态 $f(u \rightarrow v)$ 完整记录了题目规则需要的信息：当前点是 v ，上一点是 u 。因此它的转移方程和题目随机过程完全一致。

以 t 为根后，对任意非根点 v ，子树外部对 v 子树的影响只通过一个值 $B_v = f(v \rightarrow fa_v)$ 传入。后序 DP 中，我们用儿子的线性关系推出 $A_v = f(fa_v \rightarrow v)$ 关于 B_v 的线性关系，因此每个子树内部的方程都被正确维护。

自上而下时，根的儿子满足 $f(v \rightarrow t) = 0$ ，这是终点停止条件。之后每个点在已知父亲方向边界值后，解出所有儿子方向的具体状态值。因此最终所有有向边状态都满足对应的期望方程。

最后答案就是第一步之后进入的各个有向边状态的平均值，再加上第一步本身，所以算法正确。

复杂度

每个点在后序和自上而下时各处理一次，每条边只被统计常数次。

时间复杂度为 $O(n)$ ，空间复杂度为 $O(n)$ 。

所有测试数据的 n 之和不超过 $5 \cdot 10^5$ ，可以通过。

参考资料

写题解的时候发现了这个近期发布的博客（发现这个博客的时候已经来不及换题了

这题本质上与"有向边期望拆分"这一章节的结构很类似。

[随机游走：反正下一步也不知道往哪走 - 洛谷专栏](#)